Deep Learning-based Prediction of Major Page Faults in Cluster Systems

Edward Chuah[1*], Arshad Jhumka[2], Sai Narasimhamurthy[3] and Aladdin Ayesh[1]

[1*]The University of Aberdeen, Aberdeen, AB24 3FX, UK.
[2]The University of Leeds, Leeds, LS2 9JT, UK.
[3]ParTec AG, Munich, 81679, Germany.

*Corresponding author(s). E-mail(s): thuan.chuah@abdn.ac.uk;
Contributing authors: h.a.jhumka@leeds.ac.uk; sai.subr@gmail.com;
aladdin.ayesh@abdn.ac.uk;

Abstract

Page fault handling is a critical technique that increases the amount of memory available to programs, but also puts pressure on storage devices. Major page faults incur a high latency and is recognized as one of the most common causes of performance problems in cluster systems. Modern cluster systems generate a large volume of system logs and resource use data, and analyzing this data is an advocated basis for failure prediction. We set up three deep learning models including the Recurrent Neural Network (RNN), Long Short-Term Memory (LSTM) and Temporal Convolution Network (TCN). To the best of our knowledge, there is no work which predicted major page faults and page fault events in a large cluster system. In this paper, we (a) propose an approach for predicting major page faults and page fault events, and (b) evaluate the three deep learning models on real system logs and resource use data. As part of our contributions, we (a) compare the performance of the RNN, LSTM and TCN deep learning models, (b) validate our major page fault prediction approach on two large cluster systems, and (c) provide insights into major page faults and page fault events. Our work highlights empirical observations that could facilitate prediction of major page faults and page fault events in large cluster systems.

Keywords: Cluster systems, Major page faults, Deep learning, Resource use data, System logs

1 Introduction

Over the last 10 years, the capacity of spinning magnetic media storage (HDD) and flash-based memory storage (SSD) has experienced significant growth to sustain the level of performance required by scientific applications and machine learning algorithms. In this context, the storage system has become critical to meet the expectations of the end-user. The storage characteristics have been enhanced to offer higher performance and to support parallel storage functionalities including parallel input/output (I/O) paths, networked storage nodes, and separate metadata services and data services. As for performance concerns, the interface speed has grown from 98 MB/s (1G Fibre Channel) to 2,400 MB/s (Serial Attached SCSI version 4) [1]. In contrast, the read and write speeds of HDDs and SSDs have remained constant. The typical read and write speeds of an HDD are 160 MB/s and 80 MB/s

respectively, and the typical read and write speeds of a SSD is 500 MB/s and 200 MB/s respectively.

Several recent works have analyzed Linux page faults in Android mobile systems [2], investigated the performance of non-volatile memory storage under page fault and translation lookaside buffer (TLB) misses [3], proposed a mechanism that handled dynamic page faults in remote direct memory access [4], proposed a hardware software co-design approach that reduced the overhead incurred by minor page faults [5], and implemented several page replacement methods that improved processor performance under minor and major page faults [6]. In an operating system, the page fault handler is a crucial mechanism which is responsible for increasing the amount of memory available to programs [7]. However, it is also known to be the main cause of high application execution times [5, 8]. An operating system process, underlying a scientific application or machine learning algorithm activity, may experience a major page fault when it attempts to reference a page that is not present in the main memory. In this case, the operating system will stop the user process from executing, create a new page, and fetch the data for the new page from the hard disk to the main memory. Access to content on the storage device, resulting from a major page fault, may block the thread that originally required the access. For example, if a major page fault occurs on the path of displaying an earthquake simulation, the user experience will be affected by freezing frames on the display or lengthening the time to produce the final desired images. This is related to the storage device's physical characteristics which include the response times for read and write operations, fragmentation level, used and free space and to the overheads incurred by the system which include the time for scheduling I/O operations, hardware drivers complexity and filesystem inefficiency. Consequently, fault prediction schemes that can predict major page faults and page fault events in a large cluster system are highly desired.

In this paper, we use the power of deep learning models, which belong to a class of machine learning algorithms that learn complex patterns from a large volume of data, to predict major page faults and page fault events. We train the RNN, LSTM and TCN deep learning models on the system logs and resource use data to obtain the fitted models, carefully compare the performance of the three deep learning models, and apply a model validation step to ensure that the results are accurate. We validate our major page fault prediction approach on real resource use data and system logs obtained from two large cluster systems operated by the Texas Advanced Computing Center at the University of Texas at Austin. The prediction of major page faults and page fault events are new ones and have not, heretofore, been reported in our earlier papers [9–13]. The benefit of applying the RNN, LSTM and TCN deep learning models is given as follows: (a) when a major page fault is predicted by the RNN or TCN deep learning model with the highest accuracy, and (b) a page fault event is predicted by the LSTM deep learning model with the highest accuracy, it shows that a major page fault and a page fault event can be predicted by those deep learning models. When major page faults and page fault events are predicted accurately, those predictions can be used by checkpointing solutions [14, 15] to save the application state after the page fault handler has resolved the major page fault and page fault events. The main contributions of this paper are:

- We propose an approach to predict major page faults and page fault events in large cluster systems.
- We validate our major page fault prediction approach on real system logs and resource use data obtained from two large cluster systems.
- We apply a model validation step to ensure that the deep learning model's results are accurate.

Our initial assumption is that deep learning models can predict major page faults and page fault events. We compared the RNN, LSTM and TCN deep learning models and observed that (a) on System X, the TCN model predicted major page faults with the highest R^2 value on weeks 1, 2 and 4, and the RNN model predicted major page faults with the highest R^2 value on week 3, (b) on System Y, the TCN model predicted major page faults with the highest R^2 value on all the 4 weeks, and (c) on System X, the LSTM model predicted page fault events with a high R^2 value on all the 2 dates. Furthermore, we observed that the LSTM model predicted major page faults on System X with high R^2 values on all the four weeks, predicted major page faults on System Y with high R^2 values for weeks 1, 3 and 4, and obtained the

shortest average training time on the resource use data from System X and System Y.

The remainder of this paper is structured as follows. We review the related work in Section 2, and present the system model and fault model in Section 3. Then in Section 4, we describe the details of our major page fault prediction approach, and evaluate our approach in Section 5. We discuss the limitations of our approach in Section 6, and conclude with a summary and future work in Section 7.

2 Related Work

In this section, we review the related works with respect to the following criteria: (a) failure prediction frameworks that were evaluated on cluster systems, and (b) failure prediction frameworks that used different or multiple types of system logs.

In [16], the authors presented a dynamic meta-learning failure prediction engine and showed that their predictor captured failures with a low false positive rate. Then, they demonstrated the effectiveness of their predictor on real system logs from two supercomputers operated by two data centers [17]. In addition, they developed a practical failure predictor based on a genetic algorithm and showed that the location and lead time improved management of failures in an IBM BlueGene/P supercomputer significantly [18].

In [19], the authors implemented *PREdictor*, a failure prediction framework that correlated failures and forecasts the time between future failures. In [20], the authors presented a system-log preprocessing approach that categorized, filtered and correlated system events using real system logs from a Cray XT4 and an IBM BlueGene/L supercomputers, and showed that preprocessing the system logs improved failure prediction accuracy. In [21], the authors proposed a methodology that extracted sequences of events that occurred frequently and identified correlated events using different time windows. In [22] the authors combined the Event Log Signal Analyzer tool and data mining techniques, applied their method on the system logs of two supercomputers, and provided a detailed analysis of their failure prediction method. In [23], the authors presented a solution that predicted compute node soft lockups. Their method uses a decentralized online clustering algorithm that detected anomalies in the system logs. In [24], the authors proposed a novel time-based phrases extraction method that predicted node failures in Cray supercomputers, and showed that their method achieved a high recall rate and a high lead time.

In [25], the authors proposed a framework called *Desh* that uses the LSTM technique to predict node failures. In [26], the authors utilized a Frequent-Pattern Growth algorithm and an adaptive sliding window technique that predicted failures on 3 large-scale production cluster systems. In [27], the authors proposed a failure prediction framework called *PerF* that uses machine learning models to predict job failures on supercomputers. In [28], the authors proposed an attention-based failure predictor called *AtFP* that extracted meaningful features from raw system logs and predicted failures in real-time. In [29], the authors conducted a comprehensive analysis of features to characterize HPC job failures, and compared the performance of the Logistics Regression (LR), Support Vector Machine (SVM), Multilayer Perceptron (MLP), Decision Tree (DT), Random Forest (RF) and XG-Boost models to predict job failures using the scheduler logs obtained from a large HPC system. In [30], the authors proposed a novel self-supervised deep learning approach called *Clairvoyant* that predicted a sequence of log-events and identified failures that are part of the sequence using the system logs. In [31], the authors proposed a novel scalable self-supervised log-based model called *Time Machine* that predicted future log-events, failures and their location, and the lead time to a failure using the HPC system logs.

2.1 Summary

To the best of our knowledge, there is no work that predicted major page faults and page fault events in large cluster systems. A summary of the main attributes of the reviewed works is given in Table 1. Differently to the works in Table 1, we (a) develop an approach to predict major page faults and page fault events, (b) evaluate our approach on the system logs and resource use data obtained from two large cluster systems, and (c) assess the ability of the RNN, LSTM and TCN deep learning

models in predicting major page faults and page fault events.

3 System and Fault Models

In this section, we introduce the system model and major page fault model to which our major page fault prediction approach is applicable.

3.1 System Model

Our major page fault prediction approach is based on a generic model for cluster systems [32], as depicted in Fig. 1. A cluster system consists of a set of compute nodes $CN_1...CN_X$ where X is the number of compute nodes, a set of storage nodes $SN_1...SN_Y$ where Y is the number of storage nodes, a set of jobs $J_1...J_Z$ where Z is the number of jobs, and a job scheduler JS. The job scheduler JS is a software program that defines and maintains a set of non-interactive jobs that need to be automatically executed at a given time, a given frequency or when a specific event occurs. Each compute node CN_i where $1 \leq i \leq X$, storage node SN_j where $1 \leq j \leq Y$, job J_k where $1 \leq k \leq Z$ and job scheduler JS may write system logs into containers $SL_1..SL_m$ and resource use data into containers $RU_1..RU_n$. Data on the storage nodes may be stored and retrieved by each job J_k.

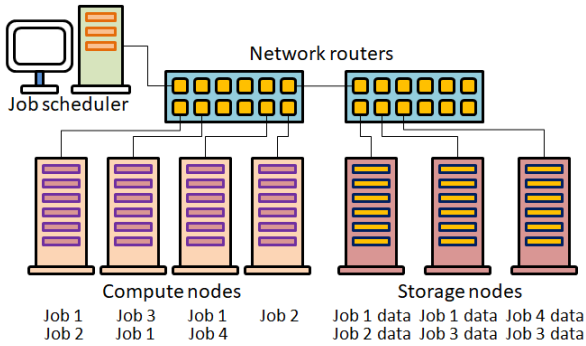


Fig. 1 Cluster system model.

3.2 Major Page Fault Model

In the development or operation of equipment, a defect is represented by an engineering model called a fault model. A fault model accurately represents the behaviour of a defect in the equipment. From the fault model, the system designer can predict the consequences of a fault in a cluster system. In this section, we describe a major page fault model to capture the relationship between the hardware and software components when a major page fault or page fault event occurs in a cluster system.

The major page fault model comprises three terms [33]: (a) fault, (b) error, and (c) failure. A fault represents the hypothesized cause of an error. An error represents a deviation from the correct service state. A failure occurs when the error leads to a loss of the service. An error is divided into two types [33]: (a) single errors, and (b) multiple errors. Single errors affect only one system component. Unlike single errors, multiple errors can propagate beyond a component's service interface and affect multiple components. An error propagation process is illustrated in Fig. 2.

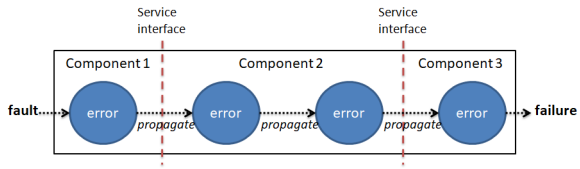


Fig. 2 An illustration of errors propagating through three system components.

For example, a bad block on a harddisk (i.e., a fault) may result in a corrupted inode (i.e., an error in component 1), generate a file access error (i.e., the first error in component 2), directory access error (i.e., the second error in component 2) and subsequently a page fault is generated by the page handling mechanism (i.e., an error in component 3) when the data or code required by the application is contained on the bad block, and causes a compute node soft lockup (i.e., a failure). The page fault, hard disk bad blocks, file access and directory access errors can occur on multiple storage and/or compute nodes and at different times. Furthermore, the log-messages which are associated with page fault events are interleaved among thousands of other log-messages. Consequently, this makes the task of predicting major page faults and page fault events in a cluster system difficult.

Table 1 Main attributes of the reviewed works

Study	Method	Logs	Focus
J. Gu *et. al.* [16]	Dynamic meta-learning	IBM BlueGene/L RAS logs	Failure prediction
Z. Lan *et. al.* [17]	Dynamic meta-learning	Multiple IBM BlueGene/L RAS logs	Failure prediction
Z. Zheng *et. al.* [18]	Genetic algorithm	IBM BlueGene/P RAS logs, Job logs	Failure prediction
S. Fu *et. al.* [19]	Covariance & stochastic models	HPC syslog traces	Failure prediction
Z. Zheng *et. al.* [20]	Event filtering & Apriori association rule mining	Cray XT4 & IBM BlueGene/L RAS logs	Failure prediction
A. Gainaru *et. al.* [21]	Frequent sequence mining	IBM BlueGene/L & LANL HPC traces	Failurep prediction
A. Gainaru *et. al.* [22]	Signal correlation & gradual itemset mining	IBM BlueGene/L & Mercury RAS logs	Failure prediction
A. Pelaez *et. al.* [23]	Decentralised online clustering	Rationalized logs	Failure prediction
A. Das *et. al.* [24]	Time-based phrases extraction	Cray HPC syslogs	Nodes failure prediction
A. Das *et. al.* [25]	LSTM	Cray HPC syslogs	Nodes failure prediction
R. Ren *et. al.* [26]	FP-growth algorithm	IBM BlueGene/L & LANL-HPC logs	Failure prediction
G. Xian *et. al.* [27]	Levenshtein distance, K-Means & PCA	HPC job logs	Job failure prediction
L. Li *et. al.* [28]	Transformer model	Spirit HPC logs	Failure prediction
J-W. Park *et. al.* [29]	LR, SVM, MLP, DT, RF, XG-Boost	Tachyon HPC job logs	Job failure prediction
K. A. Alharthi *et. al.* [30]	Transformer model & LSTM	Syslogs & Rationalized logs	Nodes failure prediction
K. A. Alharthi *et. al.* [31]	Stacked transformer model	Syslogs & Rationalized logs	Failure prediction
This paper	RNN, LSTM, TCN	Syslogs, Rationalized logs, Resource use data	Major page fault & page fault events prediction

4 Predicting Major Page Faults

In the operating system, paging is a mechanism that is used to retrieve data from the secondary storage device into the main memory. To manage the main memory, the operating system (O/S) can discard data in the main memory if the data is no longer needed or rarely used. However, at some point, a process may need to access data that is not in the main memory. When this occurs, the operating system triggers a page fault. The page fault recovery process is depicted in Fig. 3. A page fault is divided into a minor page fault or a major page fault. When the data is present in the main memory but the process is yet to obtain a logical mapping to the data, the O/S triggers a minor page fault. When the data is not present in the main memory and the process is yet to obtain a logical mapping to the data, the O/S triggers a major page fault. When a major page fault is triggered, the process is stopped, the page is copied from secondary storage into main memory, and the page table is updated to refer to the required page. Thus, a major page fault incurs a high latency, which can degrade the performance of a cluster system significantly.

Thus, our objective is to ascertain if deep learning models can predict major page faults and page fault events. By "predict", we mean a deep learning model which obtains a R^2 value

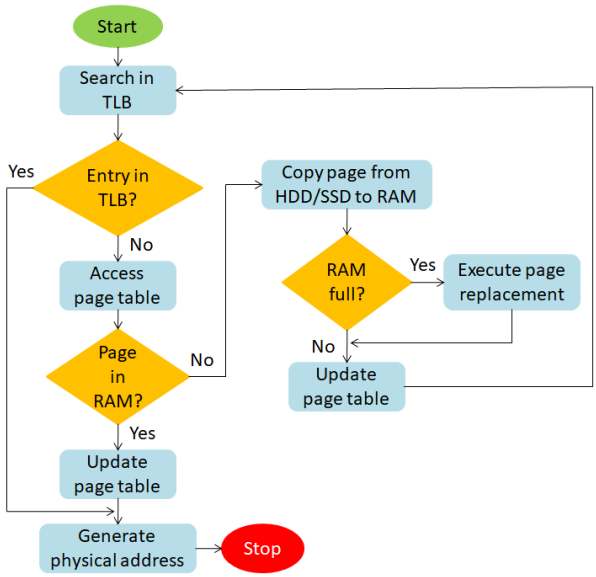


Fig. 3 A flowchart for resolving page faults.

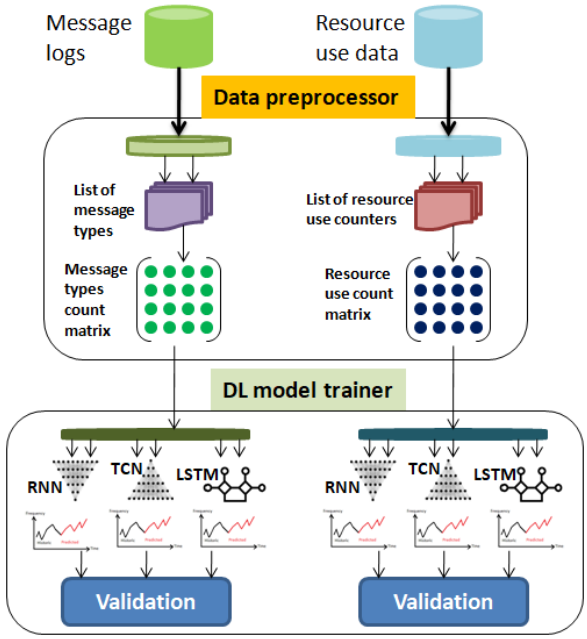


Fig. 4 The workflow of our major page fault prediction approach. It consists of a Data preprocessor module and a Deep learning model trainer module.

(coefficient-of-determination) close to 1. In this paper, we aim to predict major page faults in the resource use data and predict page fault events in the system logs. The research problem we address in this paper is as follows: Given (a) the system logs, (b) the resource use data, (c) a list of system events, (d) a list of resource use counters, and (e) a list of dates:

- Predict the number of major page faults in the resource use data by time.
- Predict the number of page fault events in the system logs by time.

As such, the major page fault prediction workflow we propose consists of 2 phases, as depicted in Fig. 4. The workflow begins with the *Data preprocessing* phase, which organizes the raw system logs and resource use data into data structures. Once the data structures are generated, the *Deep learning model training* phase applies different deep learning models to obtain the fitted model. This phase corresponds to "predicting" major page faults in the resource use data and "predicting" page fault events in the system logs. Next, we describe the details for each phase in the prediction workflow.

4.1 Data Preprocessing

The aim of data preprocessing is to transform the raw data into a standardized format so that the data can be easily parsed by data analysis algorithms [34]. To achieve this, we need to address four issues: (a) the format of the system logs generated on one cluster system is different from another cluster system, (b) the system logs contain redundant log-messages, (c) the log-messages and resource use logs are recorded separately at different times, and (d) the system logs and resource use data are captured by different monitoring tools.

4.1.1 Different system logs formats

Most large cluster systems generate system logs [35]. The system logs contain event records which are generated by the operating system, system software and filesystem. Those events include startup messages, system changes, unanticipated shutdowns, warnings and errors [36]. A log-message contains fields including the month, the date, a timestamp, a node number, a program name and a message. An example of a standard Linux Syslog [37] and a Rationalized log-message [38] is shown in Fig. 5 and Fig. 6, respectively.

From Fig. 5 and Fig. 6, we observed that (a) the fields in the Linux Syslog and Rationalized logs are organized differently, and (b) some

```
2020680 Jul 6 13:00:18 i106-308 kernel X [<%016lx>]
%s%s%s%s+0x%lx/0x%lx  18446744071562488327
do_page_fault  do_page_fault  1189  2131
```

Fig. 5 A Linux Syslog message.

```
time:1309953819
host:i106-308
jobid:2020680
prog:kernel
0:<3>try_to_free_pages
1:14
2:1189
3:0
4:ldlm_bl_08
```

Fig. 6 A Rationalized log-message.

fields are available only in the Rationalized message log. However, most message logs contain five basic fields. Those fields are the month, the date, the timestamp, the node number and the message. Therefore, we convert the Linux Syslog and Rationalized log-message into a standard formatted log which contains the following fields: `month, date, timestamp, node number, message`.

4.1.2 Redundant log-messages

In the system logs, hundreds of thousands of log-entries are generated in one day and it can contain redundant log-messages. To address this issue, we extract *Constants* from the message part in the standard formatted log. We define a *Constant* as a sequence of words that contain letters from the English alphabet, hyphen and underscore symbols, and implemented a function in our *Data preprocessor* to extract constants from the standard formatted log.

4.1.3 Different time granularities

A log-message is generated only if the print error function in the code is executed. Unlike the system logs, a resource use log is generated at fixed time intervals. As such, the time granularity of log-entries in the system logs and resource use logs is different. To address this issue, we divide the system logs and resource use logs into the same fixed time-bins. We define a time-bin as a time window of one fixed time interval.

4.1.4 Different structures of system logs

The system logs and resource use logs are recorded by different monitoring tools. As a result, the structure of the system logs and resource use logs is different. The Linux Syslog message and the Rationalized log-message contains a message part. In contrast, the resource use log contains resource use counters and their associated values. To address this issue, we generate two different types of count matrices: (a) a message types count matrix, and (b) a resource use count matrix. The message types count matrix is a single table containing the counts of all constants. Each column in the message types count matrix represents a time-bin, each row represents a constant, and each cell contains the count of a constant within a time-bin. We implemented a function in our *Data preprocessor* module to extract a list of message types and generate a message types count matrix. The process for generating a message types count matrix is given as follows:

1. Divide the message logs into time bins of 10 minutes.
2. Extract the constants from the log-messages, remove all repeated constants and store the unique constant in the list of message types.
3. For each constant in the list of message types, count the number of constants within each time bin and store the number of constants in the message types count matrix.

TACC_Stats is a resource usage monitoring infrastructure that collects system-wide performance data [39]. It is open-source and includes a web-based interface. All the values for all resource use counters are recorded using a fixed time interval of 10 minutes. The resource use counters value is set to 0 only when a node is rebooted. The resource use data contains 410 resource use counters that are divided into nine groups. The groups of resource use counters are: (1) CPU, (2) main memory, (3) virtual memory and pagefile, (4) Lustre filesystem, (5) harddisk, (6) Linux processes, (7) Infiniband network, (8) Lustre filesystem network, and (9) NUMA (Non-Uniform Memory Access). We implemented a function in our *Data preprocessor* module to extract the list of resource use counters and generate a resource use count

matrix. The resource use count matrix is a single table containing the counts of all resource use counters. Each column in the resource use count matrix represents a time-bin, each row represents a resource use counter, and each cell contains the count of a resource use counter within a time-bin. The process for generating a resource use count matrix is given as follows:

1. Divide the resource use logs into time-bins of 10 minutes.
2. Extract the resource use counter, remove all repeated resource use counters, and store the unique resource use counter in the list of resource use counters.
3. For each resource use counter in the list of resource use counters, obtain the associated value at the time-bin and store that value in the resource use count matrix.

4.2 Deep Learning Model Training

After the message types count matrix and resource use count matrix are generated, we need to predict future occurrences of major page faults and page fault events. Our *Deep Learning Model Trainer* module currently applies three deep learning models [40]: (a) Recurrent Neural Network (RNN), (b) Long Short-Term Memory (LSTM), and (c) Temporal Convolution Network (TCN). A comparison of the main features in the RNN, LSTM and TCN models is given in Table 2.

Table 2 Features in the RNN, LSTM and TCN models

Model	Architecture	Feature
RNN	Recurrent network	1. Remember input from previous time steps.
LSTM	Recurrent network	1. Process long sequences of data. 2. Handle variable-length data sequences. 3. Avoid the vanishing gradient problem.
TCN	Encoder-decoder	1. Parallel execution of data sequences. 2. Control of the model's memory size. 3. Lower memory requirements.

A RNN is a variant of the standard feedforward neural network that holds knowledge about past events [40]. An RNN uses a single type of neuron to hold information. It saves the output of a particular hidden layer and sends the result back into the hidden layer to improve on the output. LSTM is a deep learning model that learns and predicts sequential data [41]. The LSTM and RNN models are based on a recurrent network architecture, which saves the output of a particular hidden layer and sends the result back into the hidden layers to improve on the output. Differently to the RNN, an LSTM uses three types of neurons: (a) an input neuron, (b) an output neuron, and (c) a forget neuron. The input and output neurons control the input and output of the layer, respectively. The forget neuron tells a layer which input to forget. The input, output and forget neurons enable the LSTM to process long sequences of data, handle variable-length data sequences and avoid the vanishing gradient problem by controlling the flow of information and gradients through the recurrent network. The TCN is a deep learning model that uses 1-dimensional convolutions to learn and predict sequential data [42]. Differently to the RNN's and LSTM's recurrent network architecture, the TCN is based on an architecture which consists of an encoder and a decoder. Sequences of data are processed in parallel in the encoder and the output from the encoder is sent to the decoder to generate a prediction.

4.2.1 Scaling the data

Data scaling is a process that transforms the data values to fit within a specific range. The values in the message types count matrix and resource use count matrix vary in terms of their magnitude and range. For example, the number of bytes received by the Lustre filesystem is usually higher than the number of network packets dropped by the network router. Further, the range of values for the number of bytes received and the number of network packets dropped is different. Deep learning models are susceptible to overfitting during training and leads to a systematic difference between the model's estimated values and the observed values in the test data [40]. Thus, in order for a deep learning model to predict major page faults and page fault events, we need to scale the data values.

Data normalization and standardization are two standard methods for scaling the values in the data. When the data is normalized, the values are scaled to a range of $[0, n]$. However, it is challenging to identify outliers in normalized data. Unlike data normalization, when the data is standardized, the values are scaled to have a mean of 0 and a standard deviation of 1. This makes the deep learning model less sensitive to outliers in the data [40]. Therefore, we apply data standardization and scale all the values in the message types count matrix and resource use count matrix to be centered around the mean and with a unit standard deviation.

4.2.2 Model validation

Once the deep learning model is trained, we need to: (a) evaluate how close the deep learning model's predictions are to the observed values in the data, and (b) evaluate the accuracy of the deep learning model's predictions.

To evaluate how close a deep learning model's predictions are to the observed data, we apply a standard technique called residual analysis [43]. A residual is the difference between a predicted value and the actual value in the data. If the value predicted by a deep learning model is close to the actual value in the data, then the residual value will be close to 0. If the value predicted by the deep learning model is not close to the actual data value, then the residual value will not be close to 0. We obtain the residuals by taking the difference between the value predicted by the deep learning model and the actual value for all the data points.

There are three widely used metrics for evaluating the accuracy of a deep learning model's predictions. These metrics are [43]: (a) Root Mean Squared Error (RMSE), (b) Mean Absolute Error (MAE), and (c) Coefficient-of-Determination (R^2). A RMSE or MAE value ranges from 0 to infinity. An RMSE or MAE value of 0 indicates that the model predicted the observed data values perfectly. An RMSE or MAE value greater than 0 indicates that the model did not predict the observed data values perfectly. However, it is difficult to interpret a large RMSE or MAE value. Differently to the RMSE and MAE, the R^2 value ranges from 0 to 1. A R^2 value of 0 indicates that the model did not predict the observed data values perfectly. A R^2 value of 1 indicates that the model predicted the observed data values perfectly. Thus, we apply the R^2 metric to evaluate the accuracy of the RNN, LSTM and TCN models.

4.2.3 Handling inflation of the R^2 value

The R^2 value is at least weakly increasing with increases in the number of input features into a deep learning model. As a result, the R^2 value alone cannot be used to compare two deep learning models which are trained with different number of features. To compare the performance of two deep learning models, we apply a standard technique called the F-test [44]. We define the null and alternate hypotheses as: (a) the null hypothesis is that there is no difference between the full and reduced deep learning models, and (b) the alternate hypothesis is that there is a large difference between the full and reduced deep learning models. The full deep learning model is a RNN, LSTM or TCN model which is trained on all the resource use counters or all the message types. The reduced deep learning model is a RNN, LSTM or TCN model which is trained on a small number of resource use counters or a small number of message types. We implemented a function in our *Deep Learning Model Trainer* module to perform the F-test. The function uses the general linear F-statistic formula given as follows [44]: $F^* = (\frac{SSE(R)-SSE(F)}{df_R-df_F}) \div (\frac{SSE(F)}{df_F})$ where $SSE = \Sigma_{i=1}^{n}(y_i-\hat{y}_i)^2$ is the sum of squared errors, n is the number of data points, y_i is the actual value in the data, $\hat{y}_i$ is the predicted value, df_F is the degrees of freedom for the full deep learning model and df_R is the degrees of freedom for the reduced deep learning model, $df = m - k - 1$ where m is the number of resource use counters or message types and k is the number of input features in the deep learning model. When $F^* \geq 3.95$, we reject the null hypothesis in favour of the alternate hypothesis.

5 Evaluation on Large Cluster Systems

We conduct studies on two large cluster systems. System X is composed of 3,936 nodes. System Y is composed of 1,888 nodes. Systems X and Y provided batch job processing, compute resources, data storage and high-speed parallel I/O. Most

cluster systems also provide those services to their users.

To evaluate our major page fault prediction approach, we obtain 12 months worth of system logs and resource usage data from System X and System Y. However, the dates of major page faults and page fault events are not known. As such, we randomly select 4 weeks worth of resource use data and system logs from Systems X and Y. A summary of the resource use data and system logs analyzed in this study is given in Table 3 and Table 4, respectively.

Table 3 Resource use data analyzed on Systems X and Y

System	Length	Size	Quantity
X	4 weeks	142.4GB	7.2×10^8
Y	4 weeks	50.5GB	2.8×10^8

Table 4 System logs analyzed on Systems X and Y

System	Length	Size	Quantity
X	4 weeks	11.4GB	8.4×10^7
Y	4 weeks	1.52GB	1.2×10^7

Systems X and Y execute multiple jobs ranging from tectonics simulations to modeling the collision of black holes and modeling the millions of atoms in the SARS-CoV-2 virus envelope. To obtain the number of jobs executed on Systems X and Y, we identify the unique job numbers in the resource use data and system logs on Systems X and Y. There are no job numbers in the system logs on System Y. Therefore, we focus on the jobs in the resource use data on Systems X and Y and the jobs in the system logs on System X. The number of unique jobs in the resource use data and system logs are given in Table 5. From Table 5, we observed that: (a) in System X's resource use data, there are 2,574, 2,702, 3,236 and 3,811 unique jobs in weeks 1, 2, 3 and 4 respectively, (b) in System Y's resource use data, there are 11,062, 10,949, 9,013 and 9,220 unique jobs in weeks 1, 2, 3 and 4 respectively, and (c) in System X's system logs, there are 2,574, 2,702, 3,236 and 3,811 unique jobs in weeks 1, 2, 3 and 4 respectively. This shows that the resource use data and system logs are generated by multiple unique jobs on System X, and the resource use data is generated by multiple unique jobs on System Y.

Table 5 Number of unique jobs on Systems X and Y

Resource use data				
	Week			
System	1	2	3	4
X	2,574	2,702	3,236	3,811
Y	11,062	10,949	9,013	9,220
System logs				
	Week			
System	1	2	3	4
X	2,574	2,702	3,236	3,811

5.1 Model Training and Testing

To train and validate the RNN, LSTM and TCN deep learning models, we apply a standard technique called *Time Series Split (TSS) Cross-Validation (CV)* [45]. In TSS CV, the resource use count matrix and message types count matrix are divided into a training set and a test set at each iteration on the condition that the test set is always ahead of the training set. This is to ensure that the time dependency between observations in the resource use count matrix and message types count matrix are preserved. An illustration of the TSS CV is shown in Fig. 7. We implemented a function in the *DL model trainer* module to perform TSS CV. The steps for performing TSS CV are given as follows:

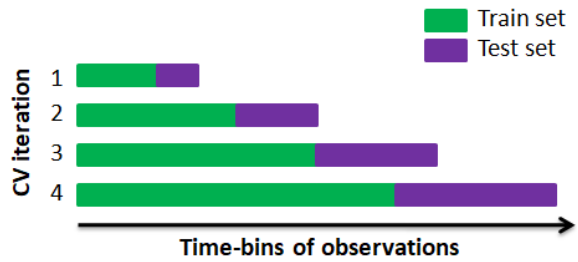


Fig. 7 An illustration of Time Series Split Cross-Validation.

1. Divide the resource use count matrix or message types count matrix into k subsets.
2. For each subset:
    1. Divide the subset's data into a training set and a test set. The training set contains the first 70% of the subset's data and the test set contains the remaining 30% of the subset's data.
    2. Train the deep learning model on the training set. Then, apply the trained model on the test set.
    3. Store the estimation error in an array and discard the model.
3. Combine the current subset and the next subset.
4. Repeat step 2 $k-1$ times.
5. Record the overall error by taking the average of estimation errors recorded in all k steps.

5.2 Phase 1: Predict Major Page Faults

When a major page fault is triggered, the `pgmajfault` resource use counter is automatically incremented. In the RNN, LSTM and TCN deep learning models, we set the `pgmajfault` resource use counter as the output to be predicted and set all the other resource use counters as input features into all the three deep learning models. Then, we train the RNN, LSTM and TCN deep learning models on the resource use count matrix and obtain the fitted model trained on all resource use counters. Furthermore, we train the RNN, LSTM and TCN deep learning models with the virtual memory (VM) resource use counters only. There are 21 virtual memory resource use counters in the resource use data on Systems X and Y. We configured the three RNN, LSTM and TCN models with a batch size of 64, 100 epochs, 2 layers and a learning rate of 0.001, and applied 10-fold time series cross-validation to train and test the three models on System X and System Y.

5.2.1 Compare the RNN, LSTM and TCN models on System X

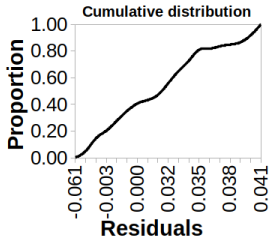
The R^2 values for the RNN, LSTM and TCN models on System X are given in Table 6. From Table 6, we observed on weeks 1 to 4 (a) the RNN model trained on all resource use counters obtained a R^2 value that ranged from 0.148 to 0.908, (b) the RNN model trained on the virtual memory resource use counters obtained a R^2 value that ranged from 0.743 to 0.993, (c) the LSTM model trained on all resource use counters obtained a R^2 value that ranged from 0.712 to 0.983, (d) the LSTM model trained on the virtual memory resource use counters obtained a R^2 value that ranged from 0.718 to 0.978, (e) the TCN model trained on all resource use counters obtained a R^2 value that ranged from 0.03 to 0.999, and (f) the TCN model trained on the virtual memory resource use counters obtained a R^2 value that ranged from 0.88 to 0.999.

Table 6 R^2 values for RNN, LSTM and TCN models on System X

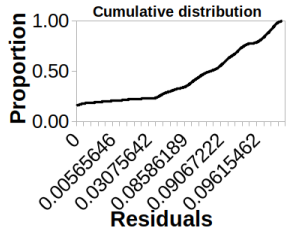
	RNN	**LSTM**	**TCN**
Week 1			
R^2 (all counters)	0.908	0.723	0.921
R^2 (VM counters)	0.983	0.803	**0.999**
Week 2			
R^2 (all counters)	0.790	0.964	**0.999**
R^2 (VM counters)	0.990	0.947	**0.999**
Week 3			
R^2 (all counters)	0.873	0.983	0.312
R^2 (VM counters)	**0.993**	0.978	0.925
Week 4			
R^2 (all counters)	0.148	0.712	0.03
R^2 (VM counters)	0.743	0.718	**0.88**

The TCN model obtained the highest R^2 value of 0.999 on weeks 1 and 2 and obtained the highest R^2 value of 0.88 on week 4, and the RNN model obtained the highest R^2 value of 0.993 on week 3, indicating that the TCN and RNN deep learning models can be used as the primary model on different weeks.

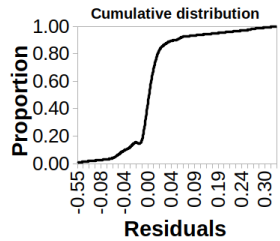
Next, we obtain the residuals in the TCN and RNN deep learning models that were trained on



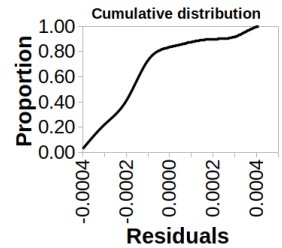
(a) Week 1, TCN model.



(b) Week 2, TCN model.



(c) Week 3, RNN model.



(d) Week 4, TCN model.

Fig. 8 Proportion of residuals in the TCN and RNN models on System X.

the virtual memory resource use counters. The proportion of residuals is shown in Fig. 8. From Fig. 8(a), we observed that the residual values in the TCN model ranged from -0.0061 to 0.0041 in week 1. From Fig. 8(b), we observed that the residual values in the TCN model ranged from 0.0056 to 0.096 in week 2. From Fig. 8(c), we observed that the residual values in the RNN model ranged from -0.55 to 0.3 in week 3. From Fig. 8(d), we observed that the residual values in the TCN model ranged from -0.0004 to 0.0004 in week 4.

On week 1, 2 and 4, all residual values in the TCN model are close to 0 and on week 3, all residual values in the RNN model are close to 0, indicating that the values predicted by the TCN and RNN models are close to the observed data values. When the R^2 value for the TCN or RNN model is close to 1 and all its residual values are close to 0, it indicates that the TCN and RNN model can be used as the primary predictor of major page faults on different weeks.

Next, we determine if the RNN and TCN models trained on all the resource use counters increased the model's fit to the observed data values. We perform the F-test on the full and reduced deep learning models and obtain the F^* value. A summary of the F^* values is given in Table 7. From Table 7, we observed that the F^* value ranged from -0.002 to 0 on weeks 1 to 4. Since $F^* < 3.95$, we fail to reject the null hypothesis.

Table 7 F-tests for the TCN and RNN models on System X

	SSE	*Degrees of Freedom*	F^*	R^2
Week 1, TCN model				
Full model	4.99	1	**-0.002**	0.921
Reduced model	3.81	389	–	**0.999**
Week 2, TCN model				
Full model	1.92	1	**0**	0.999
Reduced model	1.92	389	–	**0.999**
Week 3, RNN model				
Full model	6.39	1	**-0.002**	0.873
Reduced model	1.42	389	–	**0.993**
Week 4, TCN model				
Full model	179.16	1	**-0.002**	0.002
Reduced model	3.04	389	–	**0.88**

On weeks 1 to 4, the F^* value ranged from -0.002 to 0, indicating that there is no difference between the full and reduced deep learning models. When the F^* value is less than 3.95, the reduced model obtained the highest R^2 value and the R^2 value obtained is close to 1, the reduced TCN and RNN models can be used as the primary predictor of major page faults on different weeks.

5.2.2 Compare the RNN, LSTM and TCN models on System Y

As was done with System X, we obtain the R^2 value for the RNN, LSTM and TCN models on System Y. The R^2 values are given in Table 8. From Table 8, we observed that on weeks 1 to 4, (a) the RNN model trained on all resource use counters obtained a R^2 value that ranged from 0 to 0.982, (b) the RNN model trained on the virtual memory resource use counters obtained a R^2 value that ranged from 0 to 0.990, (c) the LSTM model trained on all resource use counters obtained a R^2 value that ranged from 0 to 0.990, (d) the LSTM model trained on the virtual memory resource use counters obtained a R^2 value that ranged from 0 to 0.983, (e) the TCN model trained on all resource use counters obtained a R^2 value that ranged from 0.403 to 0.999, and (f) the TCN model trained on the virtual memory resource use counters obtained a R^2 value that ranged from 0.865 to 0.999.

Table 8 R^2 for RNN, LSTM and TCN models on System Y

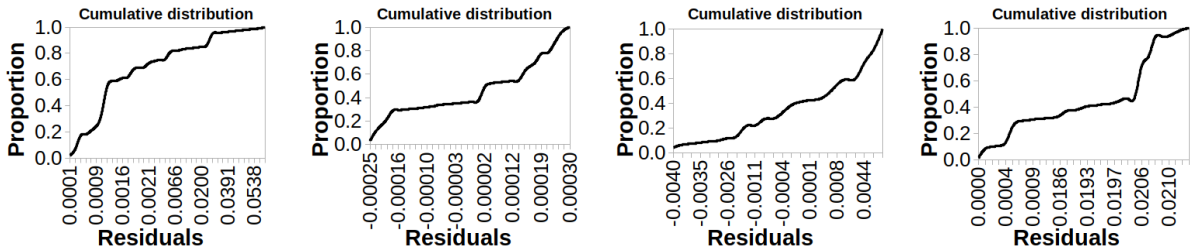
	RNN	LSTM	TCN
Week 1			
R^2 (all counters)	0.982	0.990	**0.999**
R^2 (VM counters)	0.990	0.983	**0.999**
Week 2			
R^2 (all counters)	0	0	0.403
R^2 (VM counters)	0	0	**0.865**
Week 3			
R^2 (all counters)	0.924	0.955	**0.982**
R^2 (VM counters)	0.633	0.945	0.957
Week 4			
R^2 (all counters)	0.809	0.817	**0.999**
R^2 (VM counters)	0.949	0.929	**0.999**

On weeks 1, 3 and 4, the TCN model obtained the highest R^2 value of 0.999 and on week 2, the TCN model obtained the highest R^2 value of 0.865, indicating that the TCN model can be used as the primary model on all the 4 weeks.

Next, we obtain the residuals in the TCN model trained on all resource use counters in week 3, and obtain the residuals in the TCN model trained on the virtual memory resource use counters in weeks 1, 2 and 4. The proportion of residuals is shown in Fig. 9. From Fig. 9(a), we observed that the residual values ranged from 0.0001 to 0.054 in week 1. From Fig. 9(b), we observed that the residual values ranged from -0.00025 to 0.0003 in week 2. From Fig. 9(c), we observed that the residual values ranged from -0.004 to 0.0044 in week 3. From Fig. 9(d), we observed that the residual values ranged from 0 to 0.021 in week 4.

On weeks 1 to 4, all residual values in the TCN model are close to 0, indicating that the values predicted by the TCN model are close to the observed data values. When the R^2 value for the TCN model is close to 1 and all its residual values are close to 0, it indicates that the TCN model can be used as the primary predictor of major page faults.

Next, we determine if the TCN model trained on all the resource use counters increased the model's fit to the observed data values. We perform the F-test on the full and reduced TCN model and obtain the F^* value. A summary of F^* values is given in Table 9. From Table 9, we observed that the F^* value ranged from -0.00243 to 0.117 on weeks 1 to 4. Since $F^* < 3.95$, we fail to reject the null hypothesis.



(a) Week 1. (b) Week 2. (c) Week 3. (d) Week 4.

Fig. 9 Proportion of residuals in the TCN model on System Y.

Table 9 *F*-tests for the TCN model on System Y

Week 1				
	SSE	*Degrees of Freedom*	F^*	R^2
Full model	2.08	1	**0.000025**	0.999
Reduced model	2.1	389	–	**0.999**
Week 2				
	SSE	*Degrees of Freedom*	F^*	R^2
Full model	22.03	1	**-0.00243**	0.403
Reduced model	0.712	389	–	**0.865**
Week 3				
	SSE	*Degrees of Freedom*	F^*	R^2
Full model	2.09	1	**0.117**	0.982
Reduced model	7.17	389	–	**0.957**
Week 4				
	SSE	*Degrees of Freedom*	F^*	R^2
Full model	0.719	1	**0.000032**	0.999
Reduced model	0.728	389	–	**0.999**

On weeks 1 to 4, the F^* value obtained ranged from -0.00243 to 0.117, indicating there is no difference between the full and reduced TCN models. When the F^* value is less than 3.95 and the R^2 value of the reduced model is close to 1, the reduced TCN model can be used as the primary predictor of major page faults.

5.2.3 Training time performance on resource use data

Next, we obtain the time to train the full and reduced RNN, LSTM and TCN models on the resource use data. The methods presented by the works listed in Table 1 were trained on the system logs to predict job failures or node failures. In contrast to those works, we did not train the RNN, LSTM and TCN models on the system logs to predict major page faults which lead to job or node failures. Thus, we do not compare the training times of the RNN, LSTM and TCN models with the methods listed in Table 1. The average times for training the full and reduced RNN, LSTM and TCN models on the resource use count matrices for System X and System Y are given in Table 10 and Table 11 respectively.

From Table 10, we observed that for System X the average training time of: (a) the full RNN model ranges from 86 seconds to 168 seconds, (b) the reduced RNN model ranges from 30 seconds to 43 seconds, (c) the full LSTM model ranges from 70 seconds to 89 seconds, (d) the reduced LSTM model ranges from 22 seconds to 32 seconds, (e) the full TCN model ranges from 726 seconds to 745 seconds, and (f) the reduced TCN model ranges from 708 seconds to 721 seconds.

From Table 11, we observed that for System Y the average training time of: (a) the full RNN model ranges from 28 seconds to 46 seconds, (b) the reduced RNN model ranges from 18 seconds to 28 seconds, (c) the full LSTM model ranges from 26 seconds to 32 seconds, (d) the reduced LSTM model ranges from 20 seconds to 24 seconds, (e) the full TCN model ranges from 677 seconds to 689 seconds, and (f) the reduced TCN model ranges from 668 seconds to 678 seconds.

Table 10 Average training time in seconds for the full and reduced RNN, LSTM and TCN models for System X

Model	Week			
	1	**2**	**3**	**4**
RNN (Full)	168	101	87	86
RNN (Reduced)	43	35	31	30
LSTM (Full)	89	74	72	70
LSTM (Reduced)	32	24	22	22
TCN (Full)	745	730	728	726
TCN (Reduced)	721	716	710	708

Table 11 Average training time in seconds for the full and reduced RNN, LSTM and TCN models for System Y

Model	Week			
	1	**2**	**3**	**4**
RNN (Full)	38	28	32	46
RNN (Reduced)	26	23	22	28
LSTM (Full)	32	26	26	26
LSTM (Reduced)	24	21	20	20
TCN (Full)	689	682	677	681
TCN (Reduced)	672	670	668	678

The full and reduced LSTM models obtained the shortest average training times of 22 seconds to 89 seconds on weeks 1 to 4 for System X, and obtained the shortest average training times of 20 seconds to 32 seconds on weeks 1 to 4 for System Y.

5.2.4 Summary of findings

For System X, the reduced TCN model obtained: (a) the highest R^2 value of 0.999 on weeks 1 and 2, and (b) the highest R^2 value of 0.88 on week 4. The reduced RNN model obtained the highest R^2 value of 0.993 on week 3. All the residual values in the reduced TCN model and RNN model are close to 0. These results show that the reduced TCN and RNN models are the closest fit to the observed data on different weeks. However, the time it took to train the reduced TCN model on weeks 1, 2 and 4 are 721 seconds, 716 seconds and 708 seconds respectively, and the time it took to train the reduced RNN model on week 3 is 31 seconds. While the reduced TCN and RNN models obtained the highest R^2 value on different weeks, the reduced LSTM model also obtained a high R^2 value of 0.803, 0.947, 0.978 and 0.718 on weeks 1, 2, 3 and 4 respectively. Furthermore, it took only 22 seconds to 32 seconds to train the reduced LSTM model on weeks 1 to 4. Thus, the reduced LSTM model can also be used to predict major page faults with a high accuracy on weeks 1 to 4.

For System Y, the reduced TCN model obtained: (a) the highest R^2 value of 0.999 on week 1 and 4, and (b) the highest R^2 value of 0.865 on week 2. The full TCN model obtained the highest R^2 value of 0.982 on week 3. All the residual values in the full and reduced TCN models are close to 0. These results show that the full and reduced TCN models are the closest fit to the observed data on different weeks. However, the time it took to train the reduced TCN model on weeks 1, 2 and 4 ranged from 670 seconds to 678 seconds, and the time it took to train the full TCN model on week 3 is 668 seconds. While the reduced and full TCN models obtained the highest R^2 value on different weeks, the full and reduced LSTM models also obtained a high R^2 value on weeks 1, 3 and 4. Furthermore, it took only 32 seconds and 26 seconds to train the full LSTM model on week 1 and 3 respectively, and it took only 20 seconds to train the reduced LSTM model on week 4. Thus, apart from week 2 the full and reduced LSTM models can also be used to predict major page faults with a high accuracy on weeks 1, 3 and 4.

5.3 Phase 2: Predict Page Fault Events

In the first phase of our analysis, we identified the deep learning model which predicted major page faults on Systems X and Y. We observed that (a) on System X, the TCN model predicted major page faults with the highest R^2 value on weeks 1, 2 and 4, and the RNN model predicted major page faults with the highest R^2 value on week 3, and (b) on System Y, the TCN model predicted major page faults with the highest R^2 value on all the 4 weeks. These results confirm that the RNN and TCN models predicted major page faults with the highest accuracy on Systems X and Y.

Our next objective is to determine whether the RNN, LSTM or TCN models can predict

page fault events. We scanned the system logs for log-messages containing the keywords `page fault`. We identified events that contain the constant `do_page_fault` on 2 dates on System X. However, there is no page fault events contained in the system logs on System Y. Thus, we focused on the system logs on System X. We set the `do_page_fault` event as the output to be predicted, set all the other events as input features, train the RNN, LSTM and TCN models and obtain the fitted model. Further, we train the RNN, LSTM and TCN models on events which contain the keywords `no available memory` or `out of memory`. We configured the three RNN, LSTM and TCN models with the same batch size of 64, 100 epochs, 2 layers and a learning rate of 0.001, and applied 10-fold time series cross-validation to train and test the three models on System X.

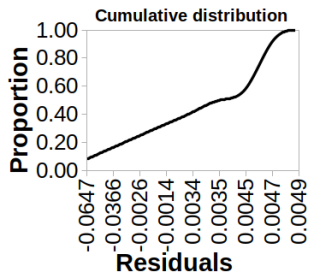
The R^2 values for the RNN, LSTM and TCN models on System X are shown in Table 12. From Table 12, we observed on days 1 and 2 (a) the RNN model trained on all system events obtained a R^2 value of 0.892 and 0.736, (b) the RNN model trained on the "no memory" events obtained a R^2 value of 0.019 and 0.226, (c) the LSTM model trained on all system events obtained a R^2 value of 0.932 and 0.893, (d) the LSTM model trained on the "no memory" events obtained a R^2 value of 0.857 and 0.997, (e) the TCN model trained on all system events obtained a R^2 value of 0.007 and 0.104, and (f) the TCN model trained on the "no memory" events obtained a R^2 value of 0.047 and 0.005.

Table 12 R^2 values for RNN, LSTM and TCN models on System X

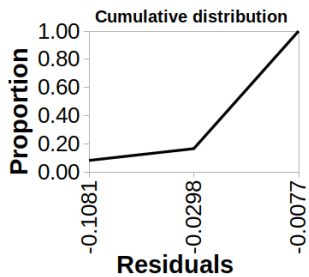
Day 1			
	RNN	**LSTM**	**TCN**
R^2 (all events)	0.829	**0.932**	0.007
R^2 (no memory events)	0.019	0.857	0.047
Day 2			
	RNN	**LSTM**	**TCN**
R^2 (all events)	0.736	0.893	0.104
R^2 (no memory) events)	0.226	0.**997**	0.005

On days 1 and 2, the LSTM model obtained the highest R^2 value of 0.932 and 0.997, indicating that the LSTM model can be used as the primary model.

Next, we obtain the residuals in the LSTM model trained on all system events on day 1 and the LSTM model trained on the "no memory" events on day 2. The proportion of residuals is shown in Fig. 10. From Fig. 10(a), we observed the residual values ranged from -0.0647 to 0.0049. From Fig. 10(b), we observed that the residual values ranged from -0.1081 to -0.0077.



(a) Day 1.



(b) Day 2.

Fig. 10 Proportion of residuals in the LSTM model on System X.

On days 1 and 2, all the residual values in the LSTM model are close to 0, indicating that the values predicted by the LSTM model are close to the observed data values. When the R^2 value for the LSTM model is close to 1 and all its residual values are close to 0, it indicates that the LSTM model can be used as the primary predictor of page fault events.

Next, we determine if the LSTM model trained on all the system events increased the model's fit to the observed data values. We perform the F-test on the full and reduced LSTM models and obtain their F^* values. A summary of the F^* values is given in Table 13. From Table 13, we observed that the F^* value on days 1 and 2 is 0.00136 and -0.0031. Since $F^* < 3.95$, we fail to reject the null hypothesis.

Table 13 F-tests for the LSTM model on System X

		Day 1		
	SSE	*Degrees of Freedom*	F^*	R^2
Full model	0.005	1	**0.0136**	0.932
Reduced model	0.07	958	–	**0.857**
		Day 2		
	SSE	*Degrees of Freedom*	F^*	R^2
Full model	0.487	1	**-0.0031**	0.893
Reduced model	0.013	319	–	**0.997**

On days 1 and 2, the F^* value obtained is 0.00136 and -0.0031 respectively, indicating there is no difference between the full and reduced LSTM models. When the F^* value is less than 3.95 and the R^2 value of the reduced model is close to 1, the reduced LSTM model can be used as the primary predictor of page fault events.

5.3.1 Training time performance on the system logs

Next, we obtain the time for training the full and reduced RNN, LSTM and TCN models on the system logs. In [31], the authors trained their system log-based self-supervised model on the raw log-sequences. In [25], the authors trained their deep learning approach on chains of log events that led to a node failure. Differently to the works in [25, 31], we did not train the RNN, LSTM and TCN models on raw log-sequences or chains of log events which lead to a system failure. Thus, we do not compare the training times of RNN, LSTM and TCN models with those works. The average times for training the full and reduced RNN, LSTM and TCN models are given in Table 14.

From Table 14, we observed that the average training time of: (a) the full RNN model is 22 seconds and 18 seconds on days 1 and 2 respectively, (b) the reduced RNN model is 18 seconds and 16 seconds on days 1 and 2 respectively, (c) the full LSTM model is 23 seconds and 20 seconds respectively, (d) the reduced LSTM model is 18 seconds and 20 seconds on days 1 and 2 respectively, (e) the full TCN model is 62 seconds and 55 seconds respectively, and (f) the reduced TCN model is 58 seconds and 52 seconds respectively.

Table 14 Average training time in seconds for the full and reduced RNN, LSTM and TCN models for System X

Model	**Day**	
	1	**2**
RNN (Full)	22	18
RNN (Reduced)	18	16
LSTM (Full)	23	20
LSTM (Reduced)	18	20
TCN (Full)	62	55
TCN (Reduced)	58	52

For System X, the full and reduced RNN and LSTM models obtained short training times that ranged between 16 seconds and 23 seconds compared to the full and reduced TCN models, which obtained longer training times that ranged between 52 seconds and 62 seconds.

5.3.2 Summary of findings

For System X, the reduced LSTM model obtained high R^2 values of 0.857 and 0.997 on days 1 and 2 respectively, and all the residual values in the reduced LSTM model are close to 0. This result shows that the reduced LSTM model is a close fit to the observed data. Furthermore, the average training time for the reduced LSTM model is 18

seconds and 20 seconds on days 1 and 2 respectively. Thus, when the R^2 value of the reduced LSTM model is close to 1, all the residual values are close to 0 and the time required to train the reduced LSTM model is low, the reduced LSTM model can be used as the main predictor of page fault events.

6 Threats to Validity

In this section, we have identified the following threats to validity (a) internal validity threats, and (b) external validity threats.

Internal validity threats are concerned with factors that might influence the results presented in this paper. We have identified the following factors which include (a) choice of dates of resource use data and system logs, and (b) validation of deep learning models. With respect to our choice of dates, we may have missed dates of major page faults or page fault events. We have addressed this issue by randomly selecting 4 weeks worth of resource usage data and system logs. On validating the deep learning models, we used the coefficient-of-determination (R^2) and residual analysis to determine how close the deep learning model's predicted values are to the observed data values. Some data centers do not release their source code due to intellectual property concerns, so we did not analyze the source code with the system logs. We did not consider other deep learning models, as they would require substantial resources beyond the scope of this paper.

External validity threats are concerned with the generalizability of the conclusions presented in this study. Our observations are based on the resource usage data and system logs of two large cluster systems, and may not apply to all large cluster systems. Having said that, the large cluster systems we used to evaluate our major page faults prediction workflow are typical of most large cluster systems. Nonetheless, we showed that our major page faults prediction approach can (a) predict major page faults with a high accuracy on System X and System Y for all the 4 weeks, and (b) predict page fault events with a high accuracy on System X for all the 2 dates. At this moment, we are gathering more resource use data and system logs from the two large cluster systems and from other cluster systems.

7 Conclusion and Future Work

An approach based on deep learning is developed to predict major page faults and page fault events in large cluster systems. We compared the RNN, LSTM and TCN deep learning models, and applied the coefficient-of-determination and residual analysis that ensured that the results are accurate. From this study, we learned that (a) the TCN model predicted major page faults with the highest R^2 value on System X on weeks 1, 2 and 4, (b) the RNN model predicted major page faults with the highest R^2 value on System X on week 3, (c) the TCN model predicted major page faults with the highest R^2 value on System Y for all the 4 weeks, and (d) the LSTM model predicted page fault events with a high R^2 value on System X for 2 dates. Further. we learned that the LSTM model predicted major page faults with high R^2 values on System X for all the four weeks, predicted major page faults with high R^2 values on System Y for weeks 1, 3 and 4, and obtained the shortest average training times on the resource use data.

In our future work, we plan to apply our major page fault prediction approach to predict other types of system faults rather than major page faults and page fault events.

Acknowledgements

We would like to thank the Texas Advanced Computing Center at the University of Texas at Austin for providing the resource usage data and system logs from their cluster systems. We would also like to thank the anonymous reviewers for their constructive feedback, which helped improve our manuscript significantly.

Declarations

Ethical Approval

Not applicable

Competing interests

All authors certify that they have no affiliations with or involvement in any organization or entity with any financial interest or non-financial interest

in the subject matter or materials discussed in this manuscript.

Authors' contributions

Edward Chuah prepared the manuscript and conducted the experiments. Arshad Jhumka, Sai Narasimhamurthy and Aladdin Ayesh reviewed and edited the manuscript.

Funding

No funding was received to assist with the preparation of this manuscript.

Availability of data and materials

The datasets analyzed during this study and the codes are available from the corresponding author on request.

References

[1] Thomasian, A.: Storage Systems, pp. 1–746. Morgan Kaufmann, ??? (2022). https://doi.org/10.1016/B978-0-32-390796-5.00024-3 . https://www.sciencedirect.com/science/article/pii/B9780323907965000243

[2] Orlando, A., Amato, P., Caraccio, D., Cinque, M., Izzi, R., Mirichigni, G., Porzio, L.: Linux page fault analysis in android systems. Microprocessors and Microsystems **66**, 10–18 (2019) https://doi.org/10.1016/j.micpro.2019.01.006

[3] Park, Y., Bahn, H.: Analysis of memory access latency considering page faults and TLB misses in NVM storage. Journal of Semiconductor Technology and Science **18**(1), 14–19 (2018) https://doi.org/10.5573/JSTS.2018.18.1.014

[4] Psistakis, A., Chrysos, N., Chaix, F., Asiminakis, M., Gianioudis, M., Xirouchakis, P., Papaefstathiou, V., Katevenis, M.: Optimized page fault handling during RDMA. IEEE Transactions on Parallel and Distributed Systems **33**(12), 3990–4005 (2022) https://doi.org/10.1109/TPDS.2022.3175666

[5] Tirumalasetty, C., Chou, C.C., Reddy, N., Gratz, P., Abouelwafa, A.: Reducing minor page fault overheads through enhanced page walker. ACM Transactions on Architecture and Code Optimization **19**(4) (2022) https://doi.org/10.1145/3547142

[6] Abraham, J.P.: An Algorithmic Approach for Optimizing Page Fault to Increase Processor Performance Along with Thread-Level Speculation, p. 178. Cochin University of Science and Technology, ??? (2017). http://hdl.handle.net/10603/231904

[7] Deitel, H.M.: Operating Systems. Addison-Wesley, ??? (1990)

[8] Mitra, S., Javagal, S., Maji, A.K., Gamblin, T., Moody, A., Harrell, S., Bagchi, S.: A study of failures in community clusters: The case of conte. In: Proceedings of the IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW), pp. 189–196 (2016). https://doi.org/10.1109/ISSREW.2016.7

[9] Chuah, E., Kuo, S.-H., Hiew, P., Tjhi, W.-C., Lee, G., Hammond, J., Michalewicz, M.T., Hung, T., Browne, J.C.: Diagnosing the root-causes of failures from cluster log files. In: Proceedings of IEEE International Conference on High-Performance Computing (HiPC), pp. 1–10 (2010). https://doi.org/10.1109/HIPC.2010.5713159

[10] Chuah, E., Jhumka, A., Narasimharmuthy, S., Hammond, J., Browne, J.C., Barth, B.: Linking resource usage anomalies with system failures from cluster log data. In: Proceedings of IEEE International Symposium on Reliable Distributed Systems (SRDS) (2013). https://doi.org/10.1109/SRDS.2013.20

[11] Chuah, E., Jhumka, A., Browne, J.C., Gurumdimma, N., Narasimharmuthy, S., Barth, B.: Using message logs and resource use data for cluster failure diagnosis. In: Proceedings of IEEE International Conference on High Performance Computing (HiPC) (2016). https://doi.org/10.1109/HiPC.2016.035

[12] Chuah, E., Jhumka, A., Alt, S., Evans, R.T., Suri, N.: Failure diagnosis for cluster systems using partial correlations. In: Proceedings of IEEE International Symposium on Parallel & Distributed Processing with Applications (ISPA) (2021). https://doi.org/10.1109/ISPA-BDCloud-SocialCom-SustainCom52081.2021.00151

[13] Chuah, E., Jhumka, A., Narasimhamurthy, S.: An empirical study of major page faults for failure diagnosis in cluster systems. Journal of Supercomputing, 1–35 (2023) https://doi.org/10.1007/s11227-023-05366-1

[14] Di, S., Bouguerra, M.S., Bautista-Gomez, L., Cappello, F.: Optimization of multi-level checkpoint model for large scale HPC applications. In: IEEE International Parallel and Distributed Processing Symposium (IPDPS), pp. 1181–1190 (2014). https://doi.org/10.1109/IPDPS.2014.122

[15] Nicolae, B., Moody, A., Gonsiorowski, E., Mohror, K., Cappello, F.: VeloC: Towards high performance adaptive asynchronous checkpointing at large scale. In: IEEE International Parallel and Distributed Processing Symposium (IPDPS), pp. 911–920 (2019). https://doi.org/10.1109/IPDPS.2019.00099

[16] Gu, J., Zheng, Z., Lan, Z., White, J., Hocks, E., Park, B.-H.: Dynamic meta-learning for failure prediction in large-scale systems: A case study. In: Proceedings of 37th International Conference on Parallel Processing (ICPP), pp. 157–164 (2008). https://doi.org/10.1109/ICPP.2008.17

[17] Lan, Z., Gu, J., Zheng, Z., Thakur, R., Coghlan, S.: A study of dynamic meta-learning for failure prediction in large-scale systems. Journal of Parallel and Distributed Computing **70**(6), 630–643 (2010) https://doi.org/10.1016/j.jpdc.2010.03.003

[18] Zheng, Z., Lan, Z., Gupta, R., Coghlan, S., Beckman, P.: A practical failure prediction with location and lead time for blue gene/p. In: 1st Workshop on Fault-Tolerance for HPC at Extreme Scale (in Conjunction with IEEE/IFIP DSN 2010) (2010). https://doi.org/10.1109/DSNW.2010.5542627

[19] Fu, S., Xu, C.-Z.: Exploring event correlation for failure prediction in coalitions of clusters. In: Proceedings of ACM/IEEE International Conference on Supercomputing (SC) (2007). https://doi.org/10.1145/1362622.1362678

[20] Zheng, Z., Lan, Z., Park, B.H., Geist, A.: System log pre-processing to improve failure prediction. In: Proceedings of IEEE/IFIP Nternational Conference on Dependable Systems and Networks (DSN) (2009). https://doi.org/10.1109/DSN.2009.5270289

[21] Gainaru, A., Cappello, F., Fullop, J., Trausan-Matu, S., Kramer, W.: Adaptive event prediction strategy with dynamic time window for large-scale HPC systems. In: Proceedings of SLAML Workshop, pp. 1–8 (2011). https://doi.org/10.1145/2038633.2038637

[22] Gainaru, A., Cappello, F., Snir, M., Kramer, W.: Fault prediction under the microscope: A closer look into HPC systems. In: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis. SC '12, pp. 77–17711 (2012). https://doi.org/10.1109/SC.2012.57

[23] Pelaez, A., Quiroz, A., Browne, J.C., Chuah, E., Parashar, M.: Online failure prediction for HPC resources using decentralized clustering. In: Proceedings of IEEE International Conference on High Performance Computing (HiPC), pp. 1–9 (2014). https://doi.org/10.1109/HiPC.2014.7116903

[24] Das, A., Mueller, F., Hargrove, P., Roman, E., Baden, S.: Doomsday: Predicting which node will fail when on supercomputers. In: Proceedings of the IEEE/ACM International Conference for High Performance Computing, Networking, Storage and Analysis (SC) (2018). https://doi.org/10.1109/SC.2018.00012

[25] Das, A., Mueller, F., Siegel, C., Vishnu, A.: Desh: Deep learning for system health prediction of lead times to failure in HPC. In:

Proceedings of ACM International Symposium on High-Performance Parallel and Distributed Computing (HPDC) (2018). https://doi.org/10.1145/3208040.3208051

[26] Ren, R., Li, J., Yin, Y., Tian, S.: Failure prediction for large-scale clusters logs via mining frequent patterns. In: Intelligent Computing and Block Chain, pp. 147–165. Springer, Singapore (2021). https://doi.org/10.1007/978-981-16-1160-5_13

[27] Xian, G., Zhang, X., Yu, J., Wang, G., Yang, W., Zhou, L., Wu, Y., Li, X., He, X.: PreF: Predicting job failure on supercomputers with job path and user behavior. Concurrency and Computation: Practice and Experience **34**(23), 7202 (2022) https://doi.org/10.1002/cpe.7202

[28] Li, L., Znati, T.: AtFP: Attention-based failure predictor for extreme-scale computing. In: Proceedings of the IEEE 13th International Conference on Reliability, Maintainability, and Safety (ICRMS), pp. 23–27 (2022). https://doi.org/10.1109/ICRMS55680.2022.9944604

[29] Park, J.-W., Huang, X., Lee, C.-H.: Analyzing and predicting job failures from HPC system log. The Journal of Supercomputing, 1–28 (2023) https://doi.org/10.1007/s11227-023-05482-y

[30] Alharthi, K.A., Jhumka, A., Di, S., Cappello, F.: Clairvoyant: A log-based transformer-decoder for failure prediction in large-scale systems. In: Proceedings of the 36th ACM International Conference on Supercomputing (ICS). Association for Computing Machinery, New York, NY, USA (2022). https://doi.org/10.1145/3524059.3532374

[31] Alharthi, K.A., Jhumka, A., Di, S., Gui, L., Cappello, F., McIntosh-Smith, S.: Time machine: Generative real-time model for failure (and lead time) prediction in HPC systems. In: Proceedings of the Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN) (2023). https://doi.org/10.1109/DSN58367.2023.00054

[32] Wellein, G., Hager, G.: Introduction to High Performance Computing for Scientists and Engineers, p. 356. CRC Press, ??? (2010)

[33] Avizienis, A., Lapire, J.-C., Randell, B., Landwehr, C.: Basic concepts and taxonomy of dependable and secure computing. IEEE Transactions on Dependable and Secure Computing **1**(1), 11–33 (2004) https://doi.org/10.1109/TDSC.2004.2

[34] Tan, P.-N., Steinbach, M., Kumar, V.: Introduction to Data Mining. Addison-Wesley, Boston (2006)

[35] He, P., Zhu, J., He, S., Li, J., Lyu, M.R.: An evaluation study on log parsing and its use in log mining. In: Proceedings of IEEE/IFIP International Conference on Dependable Systems and Networks (DSN), pp. 654–661 (2016). https://doi.org/10.1109/DSN.2016.66

[36] Chuvakin, A., Schmidt, K., Phillips, C.: Logging and Log Management, pp. 1–463. ScienceDirect, ??? (2013). https://doi.org/10.1016/C2010-0-65241-2

[37] IEEE: IEEE Std 1003.1-2001 Standard for Information Technology — Portable Operating System Interface (POSIX) Rationale (Informative), p. 310. IEEE Standards, ??? (2001)

[38] Hammond, J.L., Minyard, T., Browne, J.: End-to-end framework for fault management for open source clusters: Ranger. In: Proceedings of ACM TeraGrid Conference (2010). https://doi.org/10.1145/1838574.1838583

[39] Evans, R.T., Browne, J.C., Barth, W.L.: Understanding application and system performance through system-wide monitoring. In: Proceedings of IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW) (2016). https://doi.org/10.1109/IPDPSW.2016.145

[40] Kelleher, J.D.: Deep Learning, pp. 1–296. MIT Press, ??? (2019)

[41] Hochreiter, S., Schmidhuber, J.: Long short-term memory. Neural Computation **9**(8), 1735–1780 (1997) https://doi.org/10.1162/neco.1997.9.8.1735

[42] Lea, C., Vidal, R., Reiter, A., Hager, G.D.: Temporal convolutional networks: A unified approach to action segmentation. In: Hua, G., Jégou, H. (eds.) Computer Vision – ECCV 2016 Workshops, pp. 47–54. Springer, Cham (2016). https://doi.org/10.1007/978-3-319-49409-8_7

[43] Darlington, R.B., Hayes, A.F.: Regression Analysis and Linear Models: Concepts, Applications, and Implementation, p. 661. The Guilford Press, ??? (2016)

[44] Walpole, R.E., Myers, R.H., Myers, S.L.: Probability and Statistics for Engineers and Scientists. Prentice Hall International, Boston (1998)

[45] Kuhn, M., Johnson, K.: Applied Predictive Modeling, pp. 1–613. Springer, Cham (2018)